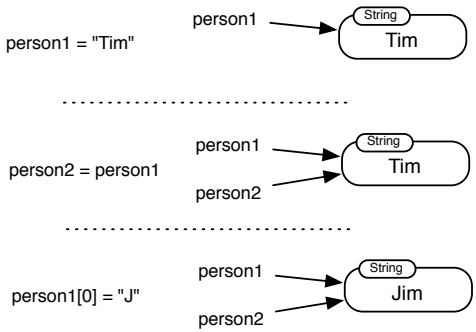


Assignment *aliases* objects, potentially giving you multiple variables that reference the same object.

But can't this cause problems in your code? It can, but not as often as you'd think (objects in Java, for example, work exactly the same way). In the previous example, for instance, you could avoid aliasing by using the dup method of String, which creates a new string object with identical contents:

```
person1 = "Tim"
person2 = person1.dup
person1[0] = "J"
puts "person1 is #{person1}"
puts "person2 is #{person2}"
```

produces:

```
person1 is Jim
person2 is Tim
```

You can also prevent anyone from changing a particular object by freezing it. Attempt to alter a frozen object, and Ruby will raise a RuntimeError exception:

```
person1 = "Tim"
person2 = person1
person1.freeze       # prevent modifications to the object
person2[0] = "J"
```

produces:

```
        from prog.rb:4:in `<main>'
prog.rb:4:in `[]=': can't modify frozen String: "Tim" (FrozenError)
```

Numbers and symbols are always frozen in Ruby, so those values are always immutable.

Reopening Classes

While we are talking about classes in Ruby we feel like we should at least mention one of the most unique features of Ruby's class structure: the ability to re-open a class definition and add new methods or variables to it at any time, even classes that are part of the third-party tools or the standard library.

In other words, if you have something like this in Ruby:

```
class Book
  attr_accessor :title
```